Module 06: False Sharing and Cache-Line Bouncing

Owner: Vishwajeet

Team: LNCT Qubit — EPAM CoE: Hardware Concurrency, Memory Models & Lock-Free Design

Date: 2026-10-09

Next handoff: Vanshraj Singh Rathore — Module 07: Compiler and CPU Reordering

Evidence labels: THEORETICAL, SOURCE-BACKED, STATICALLY REVIEWED, PENDING LOCAL EXECUTION

Status: Benchmark design and source documentation are prepared. Actual JMH measurements, charts from measured data, and hardware-counter evidence remain PENDING until execution on the target machine.

A. Scope and boundaries

In scope

- Distinguish true sharing from false sharing and explain why cache coherence operates at cache-line granularity.
- Compare independent per-thread counters in adjacent and separated array layouts using JMH.
- Specify repeatable multi-thread measurements, single-thread baseline, raw data, uncertainty, charting and environment capture.
- Explain Java layout assumptions, padding trade-offs, and what hardware counters could add.

Out of scope

- Repeating cache hierarchy/locality content owned by Module 03.
- Recreating the detailed MESI/MOESI protocol diagrams owned by Module 05.
- Treating elapsed time as direct proof of a particular coherence state or message.
- Explaining compiler/CPU memory reordering in depth; Module 07 owns that subject.

No measured result is claimed in this report. The report and diagrams are intended to make the experiment reproducible, not to replace execution evidence.

B. The false-sharing problem

False sharing occurs when two or more threads repeatedly write distinct variables that occupy the same cache line. At the Java source level, the variables may be independent. At the hardware level, coherence tracks a line-sized unit, so a write to one field can require ownership changes that affect the other field on that line. The result is usually a performance problem, not by itself a correctness bug.

True sharing versus false sharing

Property	True sharing	False sharing
Logical data	Threads access the same variable or location.	Threads update different variables/locations.
Why contention occurs	The same logical data requires coordination.	Unrelated data share a coherence line.
Can padding help?	Not by itself; synchronization semantics remain.	Often can help if layout is genuinely separated.
Correctness	Atomicity/visibility still depend on synchronization.	Padding does not repair unrelated data races.

Analogy: two people write on different sides of one board, but the entire board must be handed to the person who wants to write. The hardware similarly coordinates ownership of a whole cache line rather than individual Java fields.

C. Cache-line ownership and conceptual event sequence

A write generally requires writable ownership of the containing line. In a MESI-style teaching model, the writer needs a state that permits modification (often described as Modified). If another core has a conflicting cached copy, the coherence system coordinates a transfer or invalidation. Exact states and messages depend on the actual CPU and protocol.

Adjacent layout — illustrative only; assume a 64-byte line and 8-byte longs for the drawing. These assumptions must be checked on the target machine.

Cache line (64 B) — illustrative		
X (8 B) ← Core A writes	Y (8 B) ← Core B writes	Other bytes (48 B)

When A and B write repeatedly, writable ownership may bounce between cores even though X and Y are different variables.

Separated layout — intended design, not a proof of physical placement.

Line A (illustrative)	Line B (illustrative)
X (8 B) + unused/padding	Y (8 B) + unused/padding
Core A updates X	Core B updates Y

Conceptual timeline

```
Core A writes X → requests writable line → conflicting copy at B invalidated
Core B writes Y → requests writable line → conflicting copy at A invalidated
Core A writes X → requests writable line → conflicting copy at B invalidated
                        … repeated ownership transfers …
```

This is a conceptual event trace, not a captured snoop log. Do not claim that a specific store-buffer stall, RFO, MESI transition, or invalidation was directly observed unless suitable evidence supports that claim.

D. Java layout and padding choices

Java source declarations do not prove byte offsets or cache-line placement. The JVM may choose object layout, field order, object alignment, array base offset, compressed references, and other implementation details. Record the exact JDK and inspect layout when the report makes physical-layout claims.

1. Array stride (used by this experiment)

The benchmark places each worker counter at slot × 16 in a long[]. Sixteen longs represent 128 bytes of payload spacing. Array elements are contiguous, so the relative element spacing is predictable; this does not guarantee the array base is cache-line aligned or prove the target line size. The stride is a practical test choice, not a universal alignment guarantee.

2. Manual padding fields

Fields such as p1 through p7 can be rearranged by JVM layout rules. Class-hierarchy padding has been used by some implementations, but it is still JVM-specific and must be verified rather than assumed.

3. @Contended

jdk.internal.vm.annotation.Contended is an internal, non-stable API. Application use may require -XX:-RestrictContended and depends on JDK implementation and access restrictions. This experiment uses array striding and does not require @Contended.

Trade-offs

- Padding consumes memory and cache capacity and can affect TLB behavior.
- Padding may improve throughput, have no measurable effect, or regress performance.
- Read-only sharing is often efficient; writes that need conflicting ownership are the concern.
- A 64-byte line is an illustrative assumption, not a universal CPU fact.

E. Benchmark design

Item	Specification
Benchmark	cache.FalseSharingBench.incrementOwnedSlot
Variant parameter	stride=1 (adjacent) versus stride=16 (128-byte payload spacing)
Operation	One increment of the current JMH worker's own slot in either variant
Thread counts	Separate runs with -t 1, -t 2, -t 4, -t 8, subject to machine capacity
JMH settings	5 warmup iterations; 5 measurement iterations; 1 second each; 3 forks; throughput ops/s
Baseline	-t 1 is a low-contention/single-thread reference, not an inter-core bouncing test
Correctness	Each worker owns a unique slot; verify expected final counter values outside the timed update loop
Slot capacity	State provides 256 logical slots; do not use more than 256 workers without changing the benchmark state

The same update operation and JMH settings must be used for both layouts. Do not add shared synchronization to the timed loop unless it is intentionally part of the test.

F. Build and reproducibility procedure

Run commands from the hardware-concurrency-lab directory on a compatible JDK and Maven installation. Preserve the console output and raw JSON without editing it.

```
mvn clean test
mvn -q -DskipTests package
```

PowerShell example: separate JMH runs and raw JSON files

```
New-Item -ItemType Directory -Force results/raw/m06 | Out-Null
foreach ($threads in @(1,2,4,8)) {
  mvn -q org.codehaus.mojo:exec-maven-plugin:3.5.0:java `
    "-Dexec.mainClass=org.openjdk.jmh.Main" `
    "-Dexec.args=cache.FalseSharingBench -t $threads -rf json -rff results/raw/m06/threads-$threads.json"
}
```

If the exec plugin or JMH launch mechanism differs in the integrated project, use the project's actual supported launch method and retain the full command. A successful compilation is not a benchmark run.

Environment record — capture before interpreting results

Field	Record
JDK / JVM	java -version; vendor and complete version string
Build and benchmark	Maven version; JMH version; full command line and JVM arguments
Machine	CPU model, OS/kernel, logical/physical cores if known, cache-line size if verified
Scheduling	Affinity/pinning tool and CPU mapping if used; otherwise state not pinned
Conditions	Power mode, background load, thermal state if known, repeated campaign/fork count
Artifacts	Raw JSON path, CSV path, chart path, and any profiler output

Thread affinity is not set by the benchmark. The OS may move worker threads between logical CPUs. SMT siblings share some resources and can change the result, so record topology and placement limitations.

G. Raw output, analysis and chart

JMH JSON is the source of truth. It contains the score, score unit, error/confidence information and raw iteration data where emitted by the selected JMH version. Keep every raw file unedited and associate it with the exact run command and environment.

Minimum CSV schema

```
thread_count,stride,variant,throughput_ops_per_sec,jmh_error,
jmh_score_unit,fork_count,warmup_iterations,measurement_iterations,
jdk,cpu,os,raw_json_file
```

Chart instructions

- Use thread count on the x-axis and throughput (ops/s) on the y-axis.
- Draw separate series for adjacent stride=1 and padded stride=16.
- Include the single-thread point and error bars when supported by the output.
- Generate the chart only from collected JMH JSON/CSV; never plot template values as measurements.
- Use the module-specific chart script and check its output against the raw JMH scores.

The score error is JMH's reported uncertainty for that result; do not invent precision from a single run or substitute hand-calculated values without documenting the method.

H. Results — PENDING LOCAL EXECUTION

No actual Module 06 benchmark output is included in this report. All cells below intentionally remain pending.

Thread count	Adjacent stride=1 (ops/s)	Padded stride=16 (ops/s)	JMH error / interval	Raw JSON
1	PENDING	PENDING	PENDING	Not collected
2	PENDING	PENDING	PENDING	Not collected
4	PENDING	PENDING	PENDING	Not collected
8	PENDING	PENDING	PENDING	Not collected

Interpretation rule

If padding is faster, the result is consistent with reduced false sharing under the tested conditions, but timing alone does not prove a specific coherence transition. If results are similar or the padded version is slower, possible explanations include placement/alignment, CPU topology, scheduler movement, workload limitations, memory overhead, JIT behavior and measurement noise. Report what happened rather than forcing the expected outcome.

I. What performance counters add

On supported Linux systems, perf c2c or compatible performance-monitoring events may help identify cache-to-cache transfers, HITM-like activity, or contended line offsets. This is stronger diagnostic evidence than elapsed time alone, but it remains platform-specific.

Evidence	What it can support	What it does not automatically prove
JMH throughput and error	Performance difference for the measured setup	Exact MESI/MOESI state transitions or invalidation count
JOL/layout inspection	Object/array layout facts for the tested JVM configuration	Universal placement across JVMs or CPUs
perf c2c / compatible PMU	Potential cache-to-cache/HITM activity and line offsets	Meaning of events on every CPU; events must be validated against vendor docs
CPU/JVM environment record	Context needed to reproduce and interpret results	That affinity or thermal conditions were identical unless controlled

Record profiler version, kernel, CPU model, command line, permissions, event definitions and raw output. If counters are unavailable, state that clearly. Do not use timing as a substitute for hardware-counter evidence.

J. Common benchmark traps and misconceptions

- **Plain counters:** a thread-confined plain variable may be kept in a register or optimized; the intended repeated memory stores may not occur.
- **Naive timing:** a single System.nanoTime() loop mixes warm-up, JIT compilation, OSR, GC and scheduling noise.
- **Padding fields:** seven long fields do not guarantee a particular physical layout.
- **SMT:** running workers on sibling hardware threads may differ from placing them on separate physical cores.
- **Correctness:** false sharing is normally a performance issue; it does not make count++ atomic or remove Java Memory Model requirements.
- **Line size:** check the actual host; do not state 64 bytes as a universal rule.

K. References and source scope

- OpenJDK JMH — https://github.com/openjdk/jmh
- OpenJDK Java Object Layout (JOL) — https://openjdk.org/projects/code-tools/jol/
- Linux perf c2c manual — https://man7.org/linux/man-pages/man1/perf-c2c.1.html
- Intel 64 and IA-32 optimization documentation — https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html
- Java Language Specification — https://docs.oracle.com/javase/specs/
- Team source context: Module 04 cache coherence, Module 05 MESI/MOESI diagrams, and the master concurrency research guide.

References establish background and tooling context; exact cache-line size, event names and protocol behavior must be checked for the benchmark machine.

L. Knowledge-transfer questions

#	Question
1	Define false sharing using cache-line ownership and a MESI-style model.
2	How does true sharing differ from false sharing?
3	Why does padding by only 8 bytes often fail to separate two hot values?
4	Why does a 128-byte array stride not prove physical cache-line alignment?
5	Why might a plain long benchmark show no false sharing?
6	What are the costs and risks of padding?
7	What does @Contended do, and why is it not a stable general application API?
8	What evidence could perf c2c add, and what are its platform limitations?
9	Why can SMT placement change the result?
10	How does false sharing differ from true sharing on one atomic variable?
11	Why does elapsed time not prove a particular MESI transition?
12	How can a padded variant regress even when the design goal is reasonable?

M. Handoff packet — Vishwajeet → Vanshraj Singh Rathore

Next module: Module 07 — Compiler and CPU Reordering.

Files to pass: benchmark source, layout explanation, JMH run commands, environment record, raw JSON/CSV if collected, chart if generated from real data, profiler output if available, references, and this report.

Established here: the experiment tests performance associated with distinct counters sharing versus being separated by cache-line-sized regions. Layout and measurement results remain pending until verified on the target machine.

Do not repeat: the false-sharing theory in detail. Module 07 should instead investigate source/program order versus compiler/JIT transformations and hardware-level execution/observation order, using evidence appropriate to ordering semantics. Cache coherence does not by itself explain all Java Memory Model ordering guarantees.

N. Final self-audit

Requirement	Status	Evidence / next action
Distinguish true and false sharing	COVERED CONCEPTUALLY	Sections B and J
Adjacent vs padded Java experiment	SOURCE PREPARED	Run JMH with the same operation and multiple thread counts
Object-layout assumptions	CAVEATS DOCUMENTED	Inspect with JOL if making physical-layout claims
Raw results and chart	PENDING	Collect JMH JSON; build CSV and chart from actual scores
Hardware-counter evidence	PENDING / OPTIONAL BY PLATFORM	Run perf c2c or compatible validated profiler if available
Reproducibility details	PROCEDURE PROVIDED	Record CPU/JDK/OS/topology/affinity and exact commands
Vanshraj handoff	PROVIDED	Attach raw evidence when available
KT session	PENDING	Complete review and record feedback

Final status: STATICALLY DOCUMENTED / PENDING LOCAL EXECUTION. No TEAM-MEASURED performance claims are made in this report.